Lesson Outcomes

At the end of this chapter, student should be able to:

- ✓ Define array
- ✓ Understand requirement of array
- ✓ Know how to access elements of an array
- ✓ Write program using array
- ✓ Know how to pass array as a parameter in function

References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

A. INTRODUCTION

- Suppose you want to write a program that finds the maximum number in a list of integers. In order to know which value is the highest, the program must first read all of the numbers.
- If there are ten inputs, then you can store the data in ten variables namely `num1`, `num2`, `num3`… `num10`. Such as sequence of variable is not very practical to use.
- What if you want to find out the maximum value of `1000` numbers? It would result in a large code segment to process the data which would also be tedious and error prone.
- In programming, there is better way of implementing a collection of data items, especially if they are of the same type; that is by using the arrays.

B. DEFINITION OF AN ARRAY

- An array is a collection of a fixed number of components all of the same data type.
- A one-dimensional array is an array in which the components are arranged in a list form.
- The general form for declaring a one-dimensional array is

```
data-type arrayName[intExp];
```

Where `intExp` is any constant expression that evaluates to a positive integer. Also, `intExp` specifies the number of component in the array. For example `1, 2, 3,…, 100, ….`

While `data-type` is a value type of the elements in the array. For example `int`, `double`, `float`, and `char`.

Base Address of an Array

- The **base address** of an array is the address (memory location) of the first array component.
- For example, if `list` is a one-dimensional array, then the base address of list is the address of the component `list[0]`.
- When you pass an array as parameter, the base address of the actual array is passed to the formal parameter.

References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

- Example of `data-type` for elements of the array are:

  i. Declaration: `int myArray[5];`

  Example of elements in array with data-type `int`: `10, 24, 54, 23, 87`

  Where
  ```
  myArray[0] = 10
  myArray[1] = 24
  myArray[2] = 54
  myArray[3] = 23
  myArray[4] = 87
  ```

  ii. Declaration: float `myArray[5];`

  Example of elements in array with data-type float: `10.4, 65.4, 43.9, 54.2, 59.5`

  Where
  ```
  myArray[0] = 10.4
  myArray[1] = 65.4
  myArray[2] = 43.9
  myArray[3] = 54.2
  myArray[4] = 59.5
  ```

  iii. Declaration: char `myArray[5];`

  Example of elements in array with data-type char: `'R', 'I', 'Z', 'A', 'L'`

  OR element in array for char string: "RIZAL"

  Where
  ```
  myArray[0] = 'R'
  myArray[1] = 'I'
  myArray[2] = 'Z'
  myArray[3] = 'A'
  myArray[4] = 'L'
  ```

C. ACCESSING ARRAY COMPONENTS

- Consider the following declaration statement:

  ```
  int list[10];
  ```

- This statement declares an array list of 10 components. The components are `list[0]`, `list[1]`,…, `list[9]`. In other words, we have declared 10 variables.

`list[0]`	
`list[1]`	
`list[2]`	
`list[3]`	
`list[4]`	
`list[5]`	
`list[6]`	
`list[7]`	
`list[8]`	
`list[9]`	

References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

- The assignment statement:

```
list[5] = 34;
```

stores 34 in `list[5]`, which is the sixth component of the `array` list.

list[0]	
list[1]	
list[2]	
list[3]	
list[4]	
list[5]	34
list[6]	
list[7]	
list[8]	
List[9]	

- Suppose `i` is an `int` variable. Then the assignment statement:

```
list[3] = 63;
```

is equivalent to the assignment statements:

```
i = 3;
list[i] = 63;
```

- if `i = 4`, then the assignment statement can be written as:

```
list[4] = 58;
```

stores `58` in `list[4]`.

- What about if `i = 4` and the index of array is written as:

```
list[2 * i – 3] = 43;
```

means that stores `43` in `list[5]` because `2 * i – 3` evaluates to `5`.

- Next consider the following statements:

```
list[3] = 10;
list[6] = 35;
list[5] = list[3] + list[6];
```

The first statement stores `10` in `list [3]`, the second statement stores `35` in `list[6]`, and the third statement add the contents of `list[3]` and `list [6]` and stores the result in `list[5]`.

list[0]	
list[1]	
list[2]	
list[3]	10
list[4]	
list[5]	45
list[6]	35
list[7]	
list[8]	
List[9]	

References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

Another examples of declaration of an array and its size.

Example 1: This example is first declares a named constant and then use the value of the name constant to declare an array and specify its size.

```
const int arraySize = 10;    //declare the constant
int list[arraySize];  //declare the array name and its size
```

Example 2: This example is directly declare the name of the array and its size

```
int list[10];  //declare array with its size of 10
```

D. ARRAY INITIALIZATION DURING DECLARATION

- An array can be initialized while it is being declared.
- For example the following C++ statement declares an array, `sale` of five components and initializes these components:

  ```
  double sales[5] = {12.25, 32.50, 16.90, 23.00, 45.68};
  ```

  The values are placed between braces and separated by commas. Here `sales[0] = 12.25, sales[1] = 32.50, sales[2] = 16.90, sales[3] = 23.00, and sales[4] = 45.68`
- When initializing arrays while declaring them, it is not necessary to specify the size of the array. The size is determined by the number of initial values in the braces. However you must include the brackets following the array name. The previous statement is therefore equivalent to:

  ```
  double sales[] = {12.25, 32.50, 16.90, 23.00, 45.68};
  ```

  Although it is not necessary to specify the size of the array if it is initialized during declarations, it is a good practice to do so.

References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

Partial Initialization of Arrays During Declaration

- When declaring and initializing an array simultaneously, you do not need to initialize all the components of the array.
- This procedure is called **partial initialization of an array during declaration**. However, if you partially initialize an array during declaration, you must exercise some caution.
- Example:

```
int list[10] = {0};
```

declares list to be an array of 10 components and initializes all the components to 0.

```
int list[10] = {8, 5, 12};
```

declares list to be an array of 10 components, initializes list[0] to 8, list[1] to 5, list[2] to 12 and all other components to 0. Thus, if all the values are not specified in the initialization statement, the array components for which the values are not specified are initialized to 0.

E. ONE DIMENSIONAL ARRAY

- Some of the basic operation performed on a one-dimensional array are:
  i. Initialized
  ii. Input data
  iii. Output data stored in an array
  iv. Find the largest and / or smallest element
  v. Find the sum (for numeric)
  vi. Find the average (for numeric)
  vii. Count elements of an array

- Each of these operations requires the ability to step through the elements of the array. This is easily accomplished using a **loop**. For example, suppose that we have the following statements:

```
int list[100]; //list is an array of the size 100
int i;
```

The following for loop steps through each element of the array list, starting at the first element of list.

```
for (i = 0; i < 100; i++)    //line 1
       //process list[i]     //line 2
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

If the processing the list requires inputting data into list, the statement in Line 2 below takes the form of an input statement, such as the `cin` statement. For example, the following statement read 100 numbers from the keyboard and store the number in list.

```
for (i = 0; i < 100; i++)    //line 1
       cin >> list[i];       //line 2
```

Similarly, if processing list requires outputting the data, then the statement in Line 2 below takes the form of an output statement.

```
for (i = 0; i < 100; i++)    //line 1
       cout << list [i];     //line 2
```

Example: This example shows how loop are used to process arrays. The following declaration is used throughout this example:

```
double sale[10];
int index;
double largestSale, sum, average;
```

The first statement declares an array `sale` of 10 components with each component being of the type double. The meaning of the other statement is clear.

a. **Initializing an array**: the following loop initialized every component of the array sale to `0.0`.

```
for (index = 0; index < 10; index++)
       sale[index] = 0.0;
```

b. **Reading data into an array**: The following loop inputs the data into the array sale. For simplicity, we assume that the data is entered at the keyboard.

```
for (index = 0; index < 10; index++)
       cin >> sale[index];
```

c. **Printing an array**: The following loop outputs the array sale. For simplicity, we assume that the output goes to the screen.

```
for (index = 0; index < 10; index++)
       cout << sale[index] << " ";
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

d. **Finding the sum and average of an array**: The following C++ code finds the sum of the elements of the array sale and the average sale amount:

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
sale	12.50	8.35	19.60	25.00	14.00	39.43	35.90	98.23	66.65	35.64

```
sum = 0;
for (index = 0; index < 10; index++)
       sum = sum + sale[index];
       average =  sum / 10;
```

e. **Largest element in the array**: The following statement is to find the largest element in the array. Assume that the value element of the `sale` as follow:

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
sale	12.50	8.35	19.60	25.00	14.00	39.43	35.90	98.23	66.65	35.64

```
maxIndex = 0;
for (index = 1; index < 10; index++)
       if (sale[maxIndex] < sale[index])
       maxIndex = index;
       largestSale = sale[maxIndex];
```

maxIndex	index	sale[maxIndex]	sale[index]	sale[maxIndex] < sale[index]
0	1	12.50	8.35	False
0	2	12.50	19.60	True, maxIndex = 2
2	3	19.60	25.00	True, maxIndex = 3
3	4	25.00	14.00	False
3	5	25.00	39.43	True, maxIndex = 5
5	6	39.43	35.90	False
5	7	39.43	98.23	True, maxIndex = 7
7	8	98.23	66.65	False
7	9	98.23	35.64	False

f. **Count element in the array**: The following example is to count the number of even number and odd number in an array.

Assume that the elements of the array are as follow:

index	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
Number[index]	35	5	7	2	3	10	9	1	4	40

```
//declares array and its size
int number[10] = {35, 5, 7, 2, 3, 10, 9, 1, 4, 40};
int index;
int even = 0, odd = 0;

for (index = 0; index < 10; index++)
       if (number[index] % 2 == 0)
              even++;
       else
              odd++;
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

Example: The following example is to sort number in ascending order.

Assume that the elements of the array are as follow:

Index	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
Number[index]	35	5	7	2	3	10	9	1	4	40

```
void main()
{
	//declares array and its size
	int number[10] = {35, 5, 7, 2, 3, 10, 9, 1, 4, 40};
	int iteration, index, temp;

	//sort the element of array
	for (iteration = 0; iteration < 10; iteration++)
	{
		for (index = 0; index < 10; index++)
		{
			if(number[index] > number[index+1])
			{
				temp = number[index];
				number[index] = number[index+1];
				number[index+1] = temp;
			}
		}
	}

	//print the sorted element of array
	for (index = 0; index < 10; index++)
	{
		cout << number[index] << " ";
	}
}
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

F. DECLARE AND INITIALIZE ARRAY

- The following example shows the declaration and initialization of array.

```
#include <iostream.h>
#include <conio.h>

void main()
{
   int const array_size = 12;       //declare constant of value 10
   double sales[array_size];        //declare an array to be 10 components
   int index;                       //declare variable to read the index of array

   for (index = 0; index < array_size; index++)
       sales[index] = 0.0;   //assign 0 to all components of array sales

getch();
}
```

Explanation:

sales[0] =	0.0
sales[1] =	0.0
sales[2] =	0.0
sales[3] =	0.0
sales[4] =	0.0
sales[5] =	0.0
sales[6] =	0.0
sales[7] =	0.0
sales[8] =	0.0
sales[9] =	0.0
sales[10] =	0.0
sales[11] =	0.0

G. INPUT AND DISPLAY ELEMENTS IN ARRAY

- The following example shows the input and display element of an array.

```
#include <iostream.h>
#include <conio.h>

void main()
{
   int const array_size = 12;       //declare constant of value 10
   double sales[array_size];        //declare an array to be 10 components
   int index;                       //declare variable to read the index of array

   //initialize the components of array
   for (index = 0; index < array_size; index++)
       sales[index] = 0.0;   //assign 0 to all components of array sales
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

```
   //Get input for each component of array
   cout << "INPUT SALES" << endl;
   cout << "-------------------------------------" << endl;
   cout << "Enter sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " ;
       cin >> sales[index];  //get value for each month for every iteration
   }

   //Display components of array
   cout << "DISPLAY SALES" << endl;
   cout << "-------------------------------------" << endl;
   cout << "Sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " << sales[index] << endl;
   }

getch();
}
```

Sample output

```
INPUT SALES (RM)
-------------------------------------
Enter sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300
DISPLAY SALES (RM)
-------------------------------------
Sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

H. COUNT ELEMENT OF ARRAY

- The following example shows the way of counting the element of an array.

```
#include <iostream.h>
#include <conio.h>

void main()
{
   int const array_size = 12;       //declare constant of value 10
   double sales[array_size];        //declare an array to be 10 components
   int index;                       //declare variable to read the index of array
   int sales_LTE_1K = 0;            //to store sales less than or equal to 1000
   int sales_LTE_2K = 0;            //to store sales less than or equal to 2000
   int sales_LTE_3K = 0;            //to store sales less than or equal to 3000
   int sales_LTE_4K = 0;            //to store sales less than or equal to 4000
   int sales_GT_4K = 0;             //to store sales greater than 4000

   //initialize the components of array
   for (index = 0; index < array_size; index++)
       sales[index] = 0.0;   //assign 0 to all components of array sales

   //Get input for each component of array
   cout << "INPUT SALES" << endl;
   cout << "-----------------------------------" << endl;
   cout << "Enter sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " ;
       cin >> sales[index];  //get value for each month for every iteration
   }

   //Display components of array
   cout << "DISPLAY SALES" << endl;
   cout << "-------------------------------------" << endl;
   cout << "Sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " << sales[index] << endl;
   }

   //count components of array
   cout << "COUNT SALES RANGE" << endl;
   cout << "-------------------------------------" << endl;
   cout << "The number of sales " << endl;
   for (index = 0; index < array_size; index++)
   {
      if (sales[index] <= 1000)
               sales_LTE_1K ++;
      else if (sales[index] <= 2000)
               sales_LTE_2K ++;
      else if (sales[index] <= 3000)
               sales_LTE_3K ++;
      else if (sales[index] <=4000)
               sales_LTE_4K ++;
      else
               sales_GT_4K ++;
   }
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

```
   cout << "\t\tless than or equal to RM 1000 is " << sales_LTE_1K << endl;
   cout << "\t\tless than or equal to RM 2000 is " << sales_LTE_2K << endl;
   cout << "\t\tless than or equal to RM 3000 is " << sales_LTE_3K << endl;
   cout << "\t\tless than or equal to RM 4000 is " << sales_LTE_4K << endl;
   cout << "\t\tgreater than RM 4000 is " << sales_GT_4K << endl;

getch();
}
```

Sample output

```
INPUT SALES (RM)
---------------------------------------
Enter sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300
DISPLAY SALES (RM)
---------------------------------------
Sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300
COUNT SALES RANGE
---------------------------------------
The number of sales
                less than or equal to RM 1000 is 1
                less than or equal to RM 2000 is 6
                less than or equal to RM 3000 is 1
                less than or equal to RM 4000 is 2
                greater than RM 4000 is 2
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

I. SUMMATION AND AVERAGE VALUES OF AN ARRAY

- The following example shows the way of calculating the sum and the average of the element of an array.

```
#include <iostream.h>
#include <conio.h>

void main()
{
   int const array_size = 12;       //declare constant of value 10
   double sales[array_size];        //declare an array to be 10 components
   int index;                       //declare variable to read the index of array
   int sales_LTE_1K = 0;            //to store sales less than or equal to 1000
   int sales_LTE_2K = 0;            //to store sales less than or equal to 2000
   int sales_LTE_3K = 0;            //to store sales less than or equal to 3000
   int sales_LTE_4K = 0;            //to store sales less than or equal to 4000
   int sales_GT_4K = 0;             //to store sales greater than 4000
   double sum_sales = 0.0;          //to store the sum of sales
   double avg_sales = 0.0;          //to store the average of sales

   //initialize the components of array
   for (index = 0; index < array_size; index++)
       sales[index] = 0.0;   //assign 0 to all components of array sales

   //Get input for each component of array
   cout << "INPUT SALES" << endl;
   cout << "---------------------------------------" << endl;
   cout << "Enter sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " ;
       cin >> sales[index];  //get value for each month for every iteration
   }

   //Display components of array
   cout << "DISPLAY SALES" << endl;
   cout << "---------------------------------------" << endl;
   cout << "Sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " << sales[index] << endl;
   }

   //count components of array
   cout << "COUNT SALES RANGE" << endl;
   cout << "---------------------------------------" << endl;
   cout << "The number of sales " << endl;
   for (index = 0; index < array_size; index++)
   {
      if (sales[index] <= 1000)
               sales_LTE_1K ++;
      else if (sales[index] <= 2000)
               sales_LTE_2K ++;
      else if (sales[index] <= 3000)
               sales_LTE_3K ++;
      else if (sales[index] <=4000)
               sales_LTE_4K ++;
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

```
      else
              sales_GT_4K ++;
   }

   cout << "\t\tless than or equal to RM 1000 is " << sales_LTE_1K << endl;
   cout << "\t\tless than or equal to RM 2000 is " << sales_LTE_2K << endl;
   cout << "\t\tless than or equal to RM 3000 is " << sales_LTE_3K << endl;
   cout << "\t\tless than or equal to RM 4000 is " << sales_LTE_4K << endl;
   cout << "\t\tgreater than RM 4000 is " << sales_GT_4K << endl;

   //Summation and average value in an array
   cout << "SUMMATION AND AVERAGE VALUE OF SALES (RM)" << endl;
   cout << "-----------------------------------" << endl;
   for (index = 0; index < array_size; index++)
   {
       sum_sales = sum_sales + sales[index];      //accumulate the sum of sales in variable sum
       avg_sales = sum_sales / 12;                //calculate the average of sales
   }

   cout << "\t\tThe total sales is " << sum_sales <<endl;
   cout << "\t\tThe average sales is " << avg_sales << endl;

getch();
}
```

Sample output

```
Enter sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300

DISPLAY SALES (RM)
-----------------------------------
Sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

```
COUNT SALES RANGE
-------------------------------------
The number of sales
                less than or equal to RM 1000 is 1
                less than or equal to RM 2000 is 6
                less than or equal to RM 3000 is 1
                less than or equal to RM 4000 is 2
                greater than RM 4000 is 2

SUMMATION AND AVERAGE VALUE OF SALES (RM)
-------------------------------------
                The total sales is 28330
                The average sales is 2360.83
```

J. MAXIMUM VALUE IN AN ARRAY

- The following example shows the way of finding the maximum value of elements in an array.

```
#include <iostream.h>
#include <conio.h>

void main()
{
   int const array_size = 12;       //declare constant of value 10
   double sales[array_size];        //declare an array to be 10 components
   int index;                       //declare variable to read the index of array
   int sales_LTE_1K = 0;            //to store sales less than or equal to 1000
   int sales_LTE_2K = 0;            //to store sales less than or equal to 2000
   int sales_LTE_3K = 0;            //to store sales less than or equal to 3000
   int sales_LTE_4K = 0;            //to store sales less than or equal to 4000
   int sales_GT_4K = 0;             //to store sales greater than 4000
   double sum_sales = 0.0;          //to store the sum of sales
   double avg_sales = 0.0;          //to store the average of sales
   double max_sales;                //to store the maximum sales

   //initialize the components of array
   for (index = 0; index < array_size; index++)
       sales[index] = 0.0;   //assign 0 to all components of array sales

   //Get input for each component of array
   cout << "INPUT SALES" << endl;
   cout << "-------------------------------------" << endl;
   cout << "Enter sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " ;
       cin >> sales[index];  //get value for each month for every iteration
   }

   //Display components of array
   cout << "DISPLAY SALES" << endl;
   cout << "-------------------------------------" << endl;
   cout << "Sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " << sales[index] << endl;
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

```
   }


   //count components of array
   cout << "COUNT SALES RANGE" << endl;
   cout << "-------------------------------------" << endl;
   cout << "The number of sales " << endl;
   for (index = 0; index < array_size; index++)
   {
      if (sales[index] <= 1000)
               sales_LTE_1K ++;
      else if (sales[index] <= 2000)
               sales_LTE_2K ++;
      else if (sales[index] <= 3000)
               sales_LTE_3K ++;
      else if (sales[index] <=4000)
               sales_LTE_4K ++;
      else
               sales_GT_4K ++;
   }

   cout << "\t\tless than or equal to RM 1000 is " << sales_LTE_1K << endl;
   cout << "\t\tless than or equal to RM 2000 is " << sales_LTE_2K << endl;
   cout << "\t\tless than or equal to RM 3000 is " << sales_LTE_3K << endl;
   cout << "\t\tless than or equal to RM 4000 is " << sales_LTE_4K << endl;
   cout << "\t\tgreater than RM 4000 is " << sales_GT_4K << endl;

   //Summation and average value in an array
   cout << "SUMMATION AND AVERAGE VALUE OF SALES (RM)" << endl;
   cout << "-------------------------------------" << endl;
   for (index = 0; index < array_size; index++)
   {
       sum_sales = sum_sales + sales[index];       //accumulate the sum of sales in variable sum
       avg_sales = sum_sales / 12;                 //calculate the average of sales
   }

   cout << "\t\tThe total sales is " << sum_sales <<endl;
   cout << "\t\tThe average sales is " << avg_sales << endl;

   //Finding the maximum value in an array
   cout << "MAXIMUM SALES (RM)" << endl;
   cout << "-------------------------------------" << endl;
   max_sales = sales[0];
   for (index = 1; index < array_size; index++)
   {
       if(sales[index]> max_sales)
           max_sales = sales[index];
   }
   cout << "\t\tThe maximum sales is " << max_sales << endl;

getch();
}
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

Sample output

```
INPUT SALES (RM)
---------------------------------------
Enter sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300
DISPLAY SALES (RM)
---------------------------------------
Sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300
COUNT SALES RANGE
---------------------------------------
The number of sales
                less than or equal to RM 1000 is 1
                less than or equal to RM 2000 is 6
                less than or equal to RM 3000 is 1
                less than or equal to RM 4000 is 2
                greater than RM 4000 is 2
SUMMATION AND AVERAGE VALUE OF SALES (RM)
---------------------------------------
                The total sales is 28330
                The average sales is 2360.83
MAXIMUM SALES (RM)
---------------------------------------
                The maximum sales is 4320
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

K. MINIMUM VALUE IN AN ARRAY

- The following example shows the way of finding the minimum value of elements in an array.

```
#include <iostream.h>
#include <conio.h>

void main()
{
   int const array_size = 12;       //declare constant of value 10
   double sales[array_size];        //declare an array to be 10 components
   int index;                       //declare variable to read the index of array
   int sales_LTE_1K = 0;            //to store sales less than or equal to 1000
   int sales_LTE_2K = 0;            //to store sales less than or equal to 2000
   int sales_LTE_3K = 0;            //to store sales less than or equal to 3000
   int sales_LTE_4K = 0;            //to store sales less than or equal to 4000
   int sales_GT_4K = 0;             //to store sales greater than 4000
   double sum_sales = 0.0;          //to store the sum of sales
   double avg_sales = 0.0;          //to store the average of sales
   double max_sales;                //to store the maximum sales
   double min_sales;                 //to store the minimum sales

   //initialize the components of array
   for (index = 0; index < array_size; index++)
       sales[index] = 0.0;   //assign 0 to all components of array sales

   //Get input for each component of array
   cout << "INPUT SALES" << endl;
   cout << "-----------------------------------" << endl;
   cout << "Enter sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " ;
       cin >> sales[index];  //get value for each month for every iteration
   }

   //Display components of array
   cout << "DISPLAY SALES" << endl;
   cout << "-------------------------------------" << endl;
   cout << "Sales for the " << endl;
   for (index = 0; index < array_size; index++)
   {
       cout << "\t\tsales[month " << (index + 1) << "] = " << sales[index] << endl;
   }

   //count components of array
   cout << "COUNT SALES RANGE" << endl;
   cout << "-------------------------------------" << endl;
   cout << "The number of sales " << endl;
   for (index = 0; index < array_size; index++)
   {
      if (sales[index] <= 1000)
               sales_LTE_1K ++;
      else if (sales[index] <= 2000)
               sales_LTE_2K ++;
      else if (sales[index] <= 3000)
               sales_LTE_3K ++;
      else if (sales[index] <=4000)
               sales_LTE_4K ++;
      else
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

```
                sales_GT_4K ++;
    }

    cout << "\t\tless than or equal to RM 1000 is " << sales_LTE_1K << endl;
    cout << "\t\tless than or equal to RM 2000 is " << sales_LTE_2K << endl;
    cout << "\t\tless than or equal to RM 3000 is " << sales_LTE_3K << endl;
    cout << "\t\tless than or equal to RM 4000 is " << sales_LTE_4K << endl;
    cout << "\t\tgreater than RM 4000 is " << sales_GT_4K << endl;

    //Summation and average value in an array
    cout << "SUMMATION AND AVERAGE VALUE OF SALES (RM)" << endl;
    cout << "-----------------------------------" << endl;
    for (index = 0; index < array_size; index++)
    {
        sum_sales = sum_sales + sales[index];      //accumulate the sum of sales in variable sum
        avg_sales = sum_sales / 12;                //calculate the average of sales
    }

    cout << "\t\tThe total sales is " << sum_sales <<endl;
    cout << "\t\tThe average sales is " << avg_sales << endl;

    //Finding the maximum value in an array
    cout << "MAXIMUM SALES (RM)" << endl;
    cout << "-----------------------------------" << endl;
    max_sales = sales[0];
    for (index = 1; index < array_size; index++)
    {
        if(sales[index]> max_sales)
            max_sales = sales[index];
    }
    cout << "\t\tThe maximum sales is " << max_sales << endl;


    //Finding the minimum value in an array
    cout << endl;
    cout << "MINIMUM SALES (RM)" << endl;
    cout << "-----------------------------------" << endl;
    min_sales = sales[0];
    for (index = 1; index < array_size; index++)
    {
        if(sales[index]< min_sales)
        min_sales = sales[index];
        }
    cout << "\t\tThe minimum sales is " << min_sales << endl;

getch();
}
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

Sample output

```
Enter sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300
DISPLAY SALES (RM)
-------------------------------------
Sales for the
                sales[month 1] = 1000
                sales[month 2] = 1200
                sales[month 3] = 1300
                sales[month 4] = 1400
                sales[month 5] = 1900
                sales[month 6] = 2000
                sales[month 7] = 4320
                sales[month 8] = 4000
                sales[month 9] = 1390
                sales[month 10] = 3420
                sales[month 11] = 2100
                sales[month 12] = 4300
COUNT SALES RANGE
-------------------------------------
The number of sales
                less than or equal to RM 1000 is 1
                less than or equal to RM 2000 is 6
                less than or equal to RM 3000 is 1
                less than or equal to RM 4000 is 2
                greater than RM 4000 is 2
SUMMATION AND AVERAGE VALUE OF SALES (RM)
-------------------------------------
                The total sales is 28330
                The average sales is 2360.83
MAXIMUM SALES (RM
-------------------------------------
               The maximum sales is 4300
MINIMUM SALES (RM)
-------------------------------------
                The minimum sales is 1000
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

L. PASSING ARRAYS TO FUNCTIONS

- When passing an array parameter to a function, you place an empty [ ] behind the parameter name.
- You also need to pass the size of the array to the function. This is because there is no way the function will know the size of the array beforehand.
- Say that a function is defined to find the highest test score in a list of test scores, therefore the function may look like the following:

```
//function definition
double find_highest(double ts[ ], int ts_size)
{
	int i;
	double highest = ts[0];

	for(i = 1; i < ts_size; i++)
		if(ts[i] > highest)
		highest = ts[i];
	return highest;
}
```

- By placing the `[ ]` behind the parameter name, array parameter are always passed by reference. As such, the function can modify array parameters, and those modifications affect the array that was passed into the function. Hence, we never use an `&` (ampersand) when defining an array.
- The following program illustrates the use of array as parameters.
    - Function `read_data` reads the test score values from standard input, then function `find_highest` finds the highest test score, and the program then displays the highest test score.

```
#include <iostream.h>
#include <conio.h>

void read_data(double ts[], int ts_size)
{
	int i-;
	for(i = 0; i < ts_size; i++)
	cin >> ts[i];
}

double find_highest(double ts[], int ts_size)
{
	int i;
	double highest = ts[0];
	for(i = 1; i < ts_size; i++)
		if(ts[i] > highest)
			highest = ts[i];
	return highest;
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

```
}


void main()
{
       const int MAX_NUMSCORE = 20;
       double test_score[MAX_NUMSCORE];
       double highest_score;

       cout << "Key in the 20 Test score, one at a time, and press Enter." << endl;

       read_data(test_score, MAX_NUMSCORE);
       highest_score = find_highest(test_score,MAX_NUMSCORE);
       cout << "The highest test score is : " << highest_score;

getch();
}
```

- In `find_highest` function, to search for the highest score, variable `highest` is first initialized with the value of the first element of the array `test_score[ ]`.
- Using the value, it is compared to the rest of the elements in the array, one after another starting with the second element, as you can see the `for` loop begins with `i = 1`.
- The `if` statement compares `highest` with the value of the current element in the array.
- If the value of the array is greater than `highest`, then it is assigned to `highest` which now contains the current highest test score.
- The comparison operations continue until the loop exist, by then, `highest` will hold the maximum test score.

M. SEARCHING (SEQUENTIAL SEARCH)

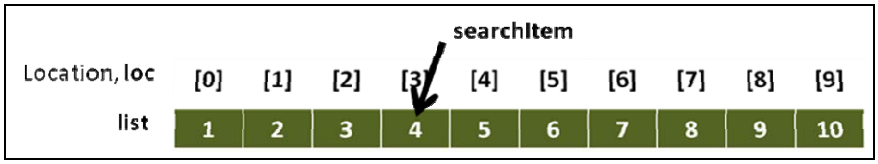
- Searching a list for a given item is one of the most common operations performed on a list.
- To search the list, you need three pieces of information:
  - i. **The list** – that is, the array containing the list**.**
  - ii. **The length of the list** (listLength <= arraySize).
  - iii. **The item for which you are searching.**
- After the search is completed,
  - iv. If the item is found, then report “success” and the location where the item was found.
  - v. If the item is not found, then report “failure”.

References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

- Example:

```
#include <iostream.h>
#include <conio.h>

void main()
{
	int list[10] = {1,2,3,4,5,6,7,8,9,10};
	int index, searchItem;
	bool found = false;
	cout << "Enter number to be searched : ";
	cin >> searchItem;
	for (index = 0; index < 10; index++)
		if (list[index] == searchItem)
		{
			found = true;
			break;
		}
	if (found)
		cout << "Number " << searchItem << " found at location " << index;
	else
		cout << "Number " << searchItem << " cannot be found in list";
getch();
}
```



Iteration	loc	list [loc]	list[loc] == searchItem	found	Next loc
1	0	1	1 == 4 is false	false	loc = 1
2	1	2	2 == 4 is false	false	loc = 2
3	2	3	3 == 4 is false	false	loc =3
4	3	4	4 == 4 is true	true	

References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

N. ARRAY OF CHARACTERS

- Character arrays are of special interest and you process them differently that you process other arrays.
- C++ provides many (predefined) function that you can use with character arrays.
- Character array: An array whose components are the type char.
- The most widely used character sets are ASCII (American Standard Code for Information Interchange) and EBCDIC (Extended Binary Coded Decimal Interchange Code).
- The first character in the ASCII character set is the null character, which is nonprintable.
- In C++, the null character is represented as '\0', a backslash followed by zero.
- The statement:

  ```
  char ch = '\0';
  ```

  Stores the null character in `ch`, where `ch` is a `char` variable.
- The null character plays an important role in processing character arrays. Because the collating sequence of the null character is 0, the null character is less than any other character in the `char` data set.
- The most commonly used term for character arrays is C-strings. However, there is a subtle difference between the two.
  - String is a sequence or zero or more characters and strings are enclosed in double quotation marks. In C++, C-strings are null terminated; that is the last character in a C-string is always the null character.

    Example: char myArray[6] = {"Hello"};

    To store "Hello" in computer memory, we need 6 memory cells of the type char.
  - A character array might not contain the null character.

    Example: char myArray[5] = {'H', 'e', 'l', 'l', 'o'};

    To store 'H', 'e', 'l', 'l', 'o' in computer memory, we only need 5 memory cells of the type char.

string.h

- This header file contains several string manipulation functions. Some of the commonly used functions are shown below.

References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam

Commonly used functions

Name	Description
`strcmp(s1, s2)`	Compares one string to another
`strcpy(s1, s2)`	Copies one string to another
`strlen(s)`	Calculates the length of a string
`strcat(s1, s2)`	Appends one string to another

Example

The following program accepts two full names; checks whether both names are equal, edit the names, get the length of the first name and concatenate two names together.

```
#include <iostream.h>
#include <string.h>
void main()
{
       char firsName[30], secName[30];
       cout << "Enter full name: " << endl;
       cin.getline(firstName, 30);
       cout << "Enter another full name: " << endl;
       cin.getline(secName, 30);

       if (strcmp(firstName, secName) == 0)
              cout << "The names are the same" << endl;
       else
              cout << "The names are different" << endl;

       strcpy(firstName, "Marissa");
       cout << "First name is now " << firstName << endl;
       cout << "The length of the first name is "
            << strlen(firstName) << endl;
       strcat(firstName, "Nurdia");
       cout << "First name is now " << firstName << endl;
}
```

Output

```
Enter full name: Nur Saidah
Enter another full name: Nurul Azua
The names are different
First name is now Marissa
The length of the first name is 7
First name is now Marissa Nurdia
```


References:
Rosenah A. H., (2009), Introduction to Programming with C++, for Scientists and Engineers, UPENA UiTM Shah Alam
Norizan M., Mazidah, P., (2006), Problem Solving with C++, UPENA UiTM Shah Alam